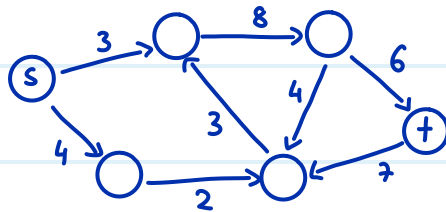


Flows

Definition Netzwerk: $N = (V, A, c, s, t)$ wobei

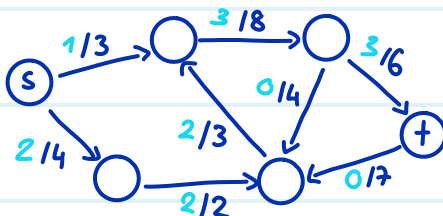
- $G = (V, A)$ ist ein gerichteter Graph (A für Arcs)
- $s \in V$ ist die source/Quelle
- $t \in V$ ist das target/Senke
- $c: A \rightarrow \mathbb{R}_0^+$ ist die Kapazitätsfunktion, die jeder Kante $e \in A$ eine Kapazität $c(e)$ zuweist



Definition Fluss: eine Funktion $f: A \rightarrow \mathbb{R}_0^+$, die jeder Kante $e \in A$ einen Flusswert $f(e)$ zuweist mit folgenden Bedingungen:

Zulässigkeit: $\forall e \in A: 0 \leq f(e) \leq c(e)$

Flusserhaltung: $\forall v \in V \setminus \{s, t\}: \underbrace{\sum_{\substack{u \in V \\ (u,v) \in A}} f(u,v)}_{\text{alles was rein fließt}} = \underbrace{\sum_{\substack{u \in V \\ (v,u) \in A}} f(v,u)}_{\text{alles was raus fließt}}$



hat Flusswert = 3

ganzzahliger Fluss: falls $\forall e \in A: f(e) \in \mathbb{Z}$

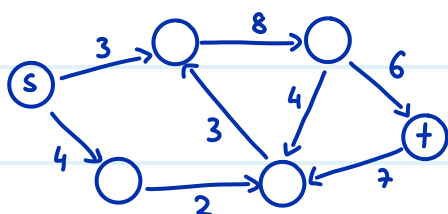
Wert des Flusses: $\text{val}(f) = \text{netoutflow}(s) = \sum_{\substack{u \in V \\ (s,u) \in A}} f(s,u) - \sum_{\substack{u \in V \\ (u,s) \in A}} f(u,s)$

was aus s raus fließt - was in s hinein fließt

Lemma
Skript
 $\text{netinflow}(t) = \sum_{\substack{u \in V \\ (u,t) \in A}} f(u,t) - \sum_{\substack{u \in V \\ (t,u) \in A}} f(t,u)$

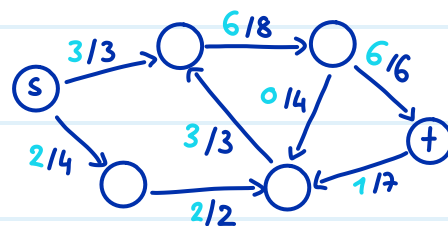
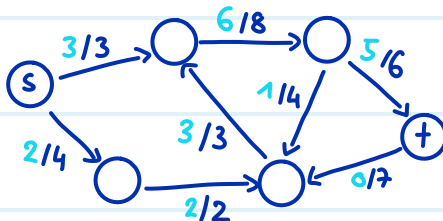
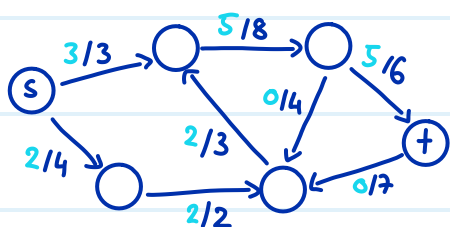
was in t hinein fließt - was aus t raus fließt

Aufgabe: bestimme den maximalen Flusswert für folgendes Netzwerk:



MaxFlow = 5

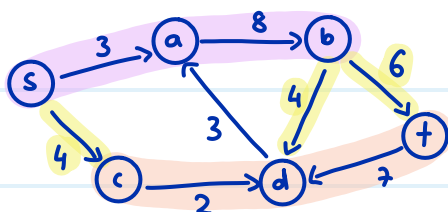
Finde drei verschiedene Flüsse in diesem Netzwerk, die alle den maximalen Wert haben



s-t-Schnitt: eine Partition (S, T) von V sodass $s \in S$ und $t \in T$
 das heisst $S \cup T = V$
 und $S \cap T = \emptyset$

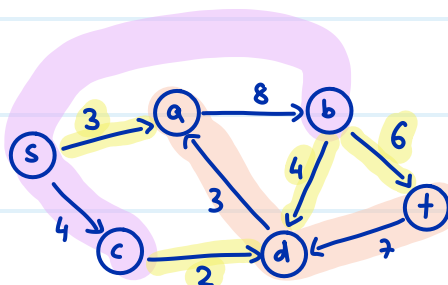
Kapazität eines s-t-Schnitts: $cap(S, T) = \sum_{(u,v) \in (S \times T) \cap A} c(u,v)$
 alle Kanten die von S zu T gehen

Bsp



Sei $S = \{s, a, b\}$ und $T = \{t, c, d\}$

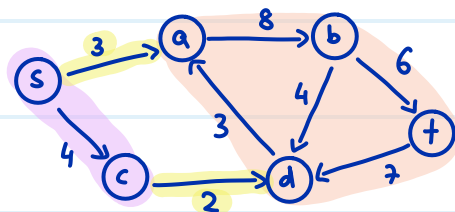
dann ist $cap(S, T) = 4 + 3 + 6 = \underline{\underline{14}}$



Sei $S = \{s, b, c\}$ und $T = \{t, a, d\}$

Berechne $cap(S, T) = 3 + 2 + 6 + 4 = \underline{\underline{15}}$

Finde einen s-t-Schnitt mit $cap(S, T) = 5$



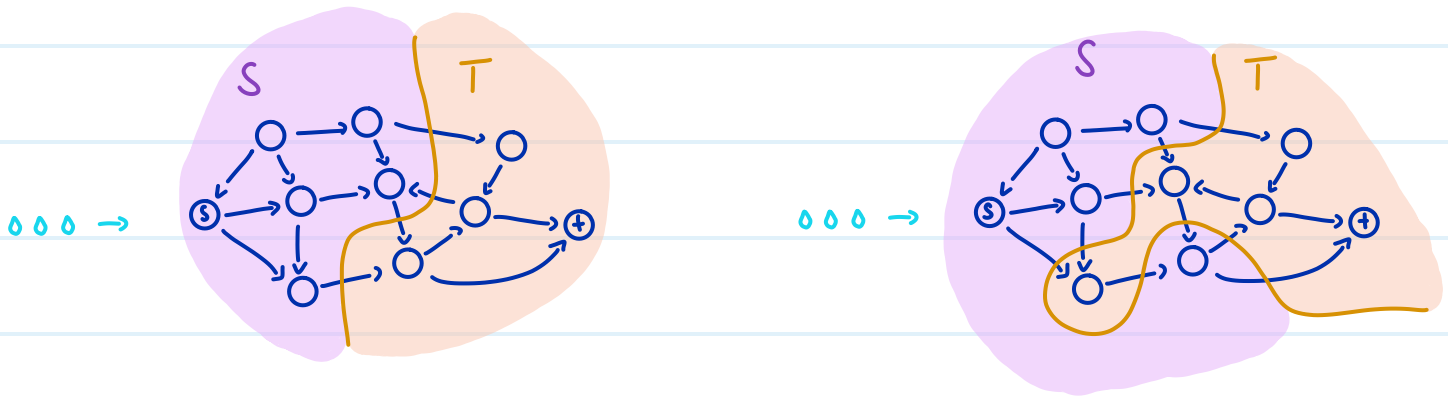
Sei $S = \{s, c\}$ und $T = \{t, a, b, d\}$

dann ist $cap(S, T) = 3 + 2 = \underline{\underline{5}}$

Lemma 3.8. Für einen beliebigen Fluss und einen beliebigen s-t-Schnitt (S, T) gilt:

$$val(f) \leq cap(S, T)$$

Intuition: Jeder einzelne "Tropfen" muss irgendwann die Grenzlinie von S nach T überqueren



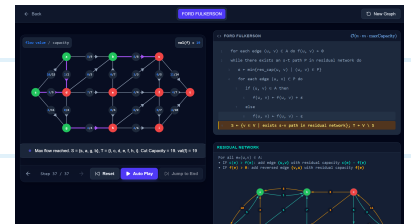
Maxflow-Mincut Theorem: $\max_{\text{Fluss } f} \text{val}(f) = \min_{(S,T) \text{ s-t-Schnitt}} \text{cap}(S,T)$

$\Rightarrow$ um zu beweisen, dass ein Fluss f maximal ist, reicht es, einen s-t-Schnitt zu finden sodass $\text{val}(f) = \text{cap}(S,T)$

Beweis mittels Ford-Fulkerson Algorithmus:

FORD-FULKERSON(V, A, c, s, t)

- | | |
|--|---|
| 1: $f \leftarrow 0$ | $\triangleright$ Fluss konstant 0 |
| 2: while $\exists$ s-t-Pfad P in (V, A_f) do | $\triangleright$ augmentierender Pfad |
| 3: Erhöhe den Fluss entlang P | $\triangleright$ wie in Beweis zu Satz 3.11 |
| 4: return f | $\triangleright$ maximaler Fluss |



- Laufzeit $O(nmU)$ falls alle Kapazitäten ganzzahlig sind und U die maximale Kapazität ist (ähnliche Idee wie bei Hopcroft-Karp)

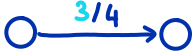
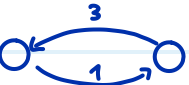
Restnetzwerk: Sei $N = (V, A, c, s, t)$ ein Netzwerk und f ein Fluss.

Dann ist $N_f = (V, A_f, r_f, s, t)$ das Restnetzwerk, wobei

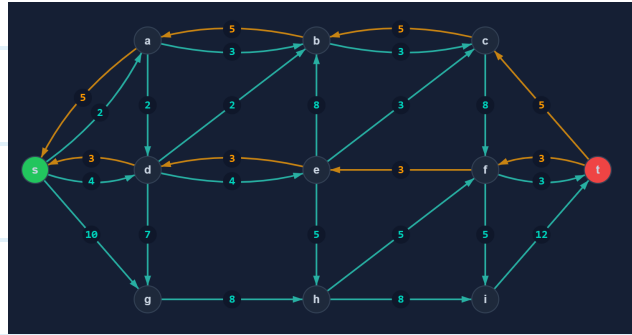
- für jede noch nicht vollständig ausgelastete (d.h. $f(e) < c(e)$) Kante $e \in A$ fügen wir die Kante e zu A_f hinzu mit der noch übrigen Restkapazität $r_f(e) := c(e) - f(e)$
- für jede Kante $e \in A$ auf der etwas fließt (d.h. $f(e) > 0$) fügen wir eine umgekehrte Kante e^{opp} zu A_f hinzu mit dem fließenden Wert $r_f(e^{opp}) := f(e)$
- Wir nehmen an, dass es in N keine einander entgegen gerichtete Kanten gibt

Intuitiv sagt uns das Restnetzwerk, wie wir den aktuellen Fluss verändern können

Aufgabe:

Kante in N	Kante(n) in N_f
	
	
	
	
	

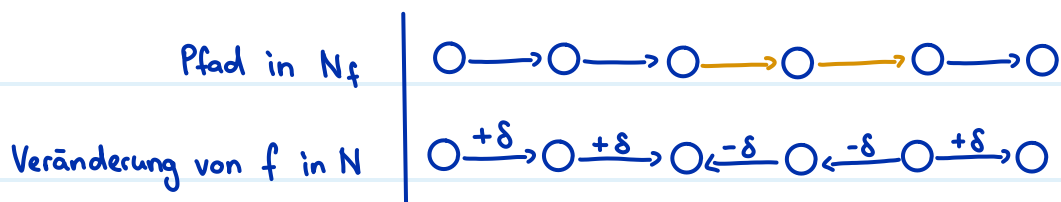
Augmentierende Pfade: gerichtete Pfade von s nach t im Restnetzwerk



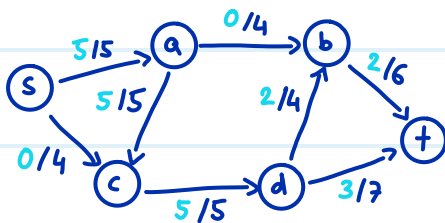
$\delta :=$ minimale Restkapazität einer Kante des augmentierenden Pfades

Für eine Kante e des augmentierenden Pfades P ändern wir f wie folgt.

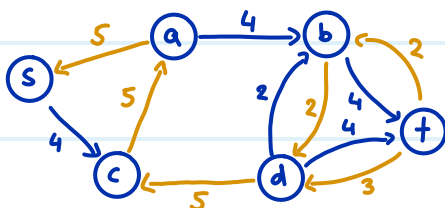
$$f(e) = \begin{cases} f(e) + \delta & \text{falls die Kante in } P \text{ in dieselbe Richtung zeigt wie in } N \\ f(e) - \delta & \text{sonst} \end{cases}$$



Aufgabe: Gegeben sind das folgende Netzwerk und der Fluss:



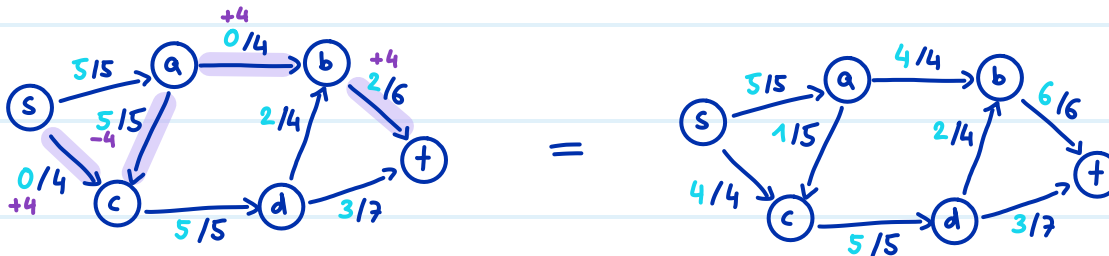
1) Zeichne das Restnetzwerk ein.



2) Finde einen augmentierenden Pfad: (s, c, a, b, t)

3) Bestimme die minimale Restkapazität einer Kante des augmentierenden Pfades: $\delta = 4$

4) Augmentiere den Fluss mit diesem Pfad.



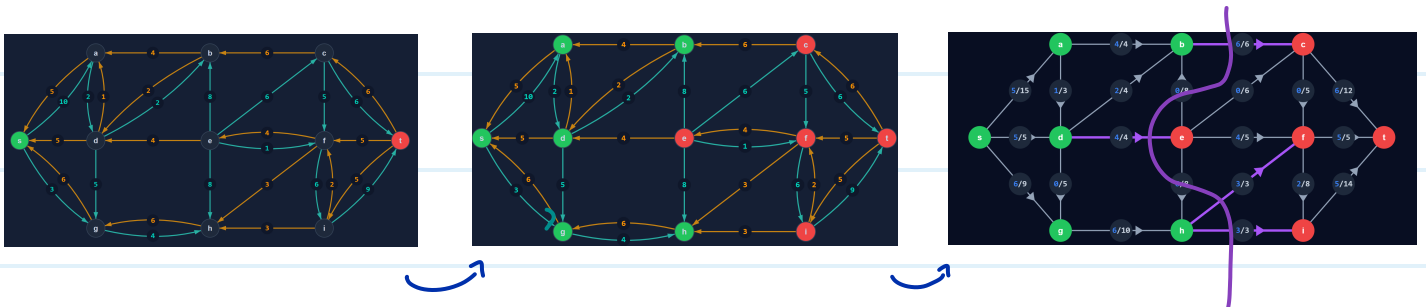
Satz 3.11: f ist maximal $\Leftrightarrow$ es gibt keinen gerichteten s - t -Pfad in N_f

(analog zu Satz von Berge)

Wenn f maximal ist, dann gilt $\text{cap}(S, T) = \text{val}(f)$

für $S = \{v \in V \mid v \text{ ist von } s \text{ aus erreichbar in } N_f\}$

und $T = V \setminus S$



Anwendungen: Verkehr, Strom-/Wasserversorgung, Internet, Truppenverschiebung, ... (sehr viele)

Für die Prüfung: höchstwahrscheinlich keine Netzwerk-Beweise

höchstwahrscheinlich eine Code Expert Modellierungsaufgabe

Aufgaben:

N Familien nehmen an einem Abendessen teil. Die i -te Familie besteht aus m_i Mitgliedern. Es stehen T Tische zur Verfügung, wobei der j -te Tisch genau s_j viele Stühle hat. Um den sozialen Austausch zu fördern, dürfen keine zwei Mitglieder derselben Familie am selben Tisch sitzen.

Erstelle ein Netzwerk, mit dem sich bestimmen lässt, ob alle Familienmitglieder unter Einhaltung dieser Regel untergebracht werden können.

Sei $N = (V, A, c, s, t)$ ein Netzwerk, wobei

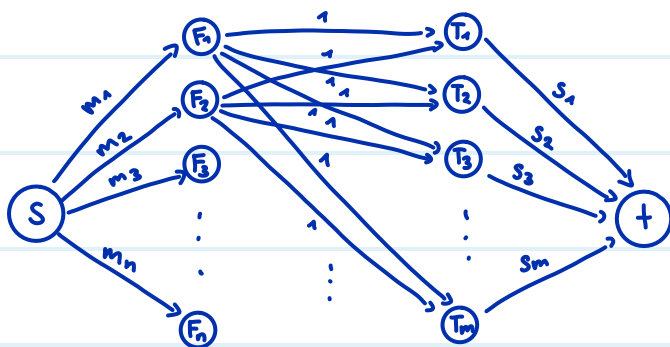
$$V = \{ s, t, \underbrace{F_1, F_2, \dots, F_n}_{\text{ein Knoten pro Familie}}, \underbrace{T_1, T_2, \dots, T_m}_{\text{ein Knoten pro Tisch}} \}$$

• A hat folgende Kanten

(s, F_i) mit Kapazität $m_i \quad \forall 1 \leq i \leq n$

(F_i, T_j) mit Kapazität 1 $\quad \forall 1 \leq i \leq n \quad \forall 1 \leq j \leq m$

(T_j, t) mit Kapazität $s_j \quad \forall 1 \leq j \leq m$



$$\text{Zuteilung möglich} \iff \max_{\text{Fluss } f} \text{val}(f) = \underbrace{\sum_{i=1}^n m_i}_{\text{alle Menschen}}$$

In einer Fabrik müssen n Aufträge bis in F Tagen abgeschlossen werden. Jeder Auftrag benötigt genau einen Arbeitstag. Der i -te Auftrag kann jedoch nur zwischen einem Starttag s_i und einem Endtag e_i ausgeführt werden. Ihnen stehen m Maschinen zur Verfügung, und jede Maschine kann höchstens einen Auftrag pro Tag bearbeiten. Erstelle ein Netzwerk, um zu prüfen, ob alle n Aufträge bis zur Frist abgeschlossen werden können.

Sei $N = (V, A, c, s, t)$ ein Netzwerk, wobei

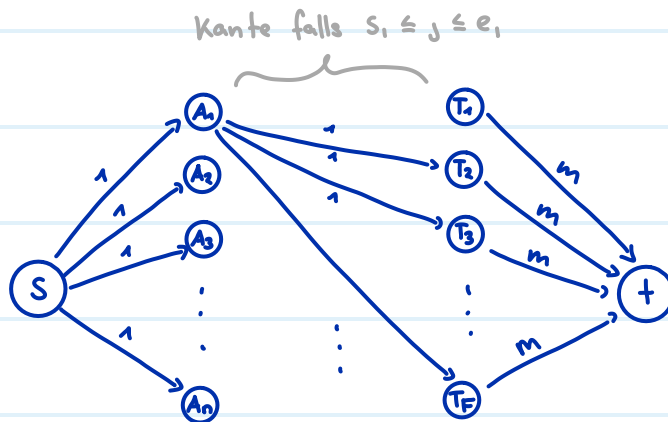
$$V = \{ s, t, \underbrace{A_1, A_2, \dots, A_n}_{\text{ein Knoten pro Auftrag}}, \underbrace{T_1, T_2, \dots, T_F}_{\text{ein Knoten pro Tag}} \}$$

• A hat folgende Kanten

(s, A_i) mit Kapazität 1 $\forall 1 \leq i \leq n$

(A_i, T_j) mit Kapazität 1 $\forall 1 \leq i \leq n \forall 1 \leq j \leq F$ mit $s_i \leq j \leq e_i$

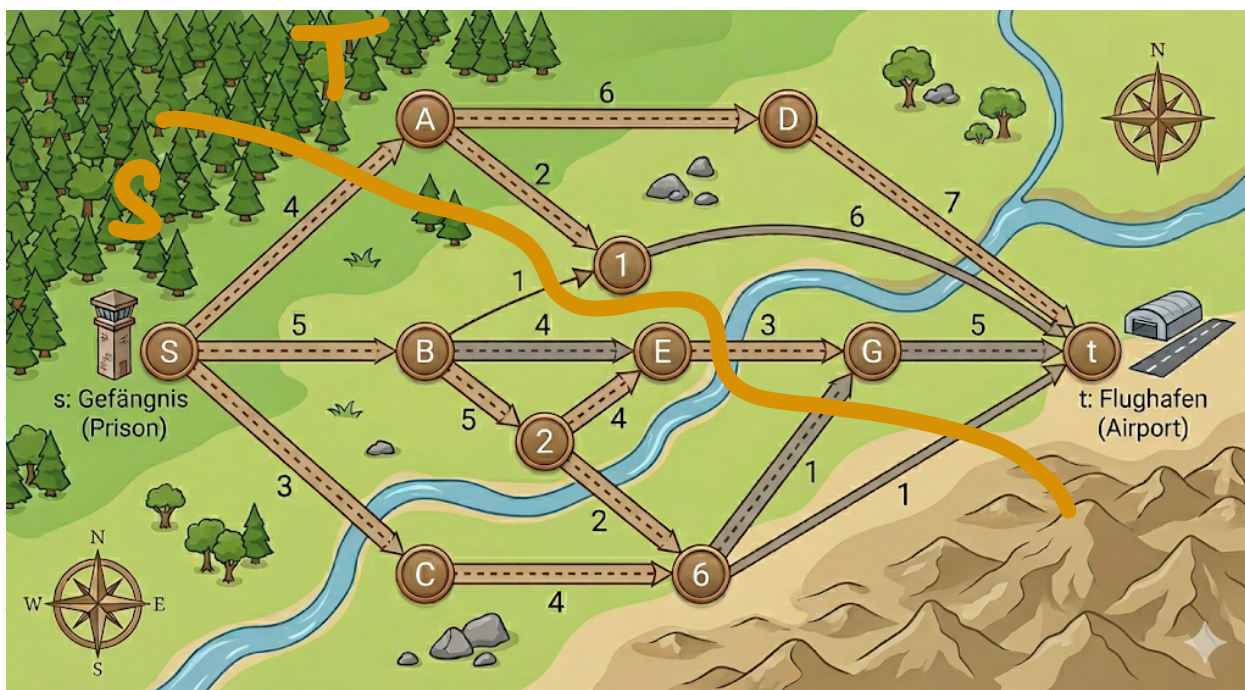
(T_j, t) mit Kapazität m $\forall 1 \leq j \leq F$



Einhaltung der Frist ist möglich $\Leftrightarrow \max_{\text{Fluss}} \text{val}(f) = \underbrace{n}_{\text{alle Aufträge}}$

Ein Dieb bricht aus dem Gefängnis s aus, und möchte zum Flughafen t flüchten. Dabei benutzt er das Strassennetz. Die Polizei verfügt über 10 Polizisten, die sie auf Strassen platzieren können. Für jede Strasse ist angegeben, wie viele Polizisten benötigt werden, um diese zu blockieren.

Ist es möglich, mit 10 Polizisten sicherzustellen, dass der Dieb nicht zum Flughafen gelangt?



Ja es ist möglich, weil $\max_{\text{Fluss } f} \text{val}(f) = 10 \Rightarrow \text{minimaler Schnitt} = 10$

Flow Aufgaben

Ein Datennetzwerk $N = (V, A, c, s, t)$ besteht aus n Servern und einigen gerichteten Kabelverbindungen dazwischen. Ein Hauptserver s möchte Datenpakete an einen Zielserver t senden. Jedes Kabel kann maximal c Pakete pro Sekunde weiterleiten. Allerdings haben auch die Server selbst eine begrenzte Rechenleistung: Der Server i kann pro Sekunde insgesamt maximal v_i Datenpakete verarbeiten und weiterleiten (egal woher sie kommen und wohin sie gehen).

Modelliere den maximalen Datenfluss von s nach t als ein maxflow Problem.

Wir erstellen ein modifiziertes Netzwerk $N' = (V', A', c', s', t')$ wobei

$$V' = \{i_{in} \mid i \in V\} \cup \{i_{out} \mid i \in V\}$$

wir teilen jeden Knoten in einen "in" und einen "out" Knoten auf

$$A' = \{(i_{out}, j_{in}) \mid (i, j) \in A\} \cup \{(i_{in}, i_{out}) \mid i \in V\}$$

"echte" Kanten zwischen
zwei Servern

Verarbeitungskanten

$$c'(x, y) = \begin{cases} v_i & \text{falls } x = i_{in} \text{ und } y = i_{out} \\ c & \text{sonst} \end{cases}$$

→ Verarbeitungskanten

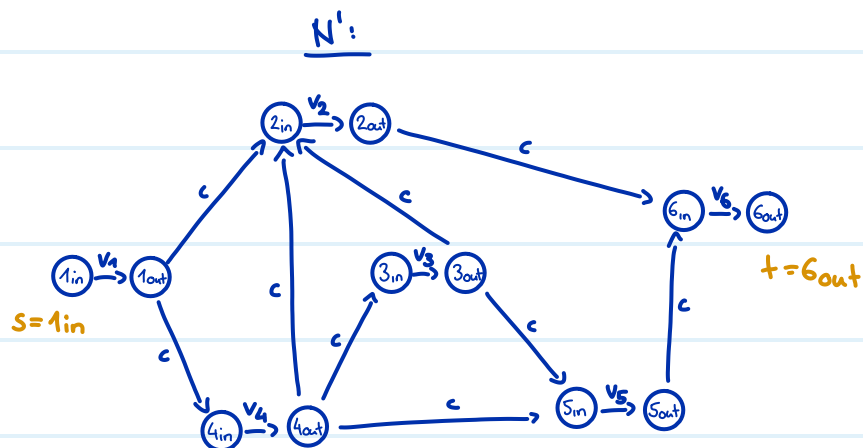
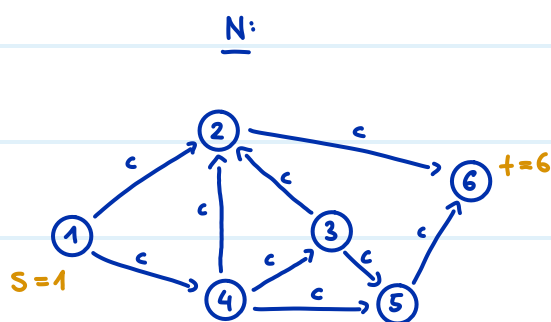
→ "echte" Kanten

$$s' = s_{in}$$

$$t' = t_{out}$$

Der maximale Datenfluss im Datennetzwerk entspricht dem maxflow in N'

Beispiel:

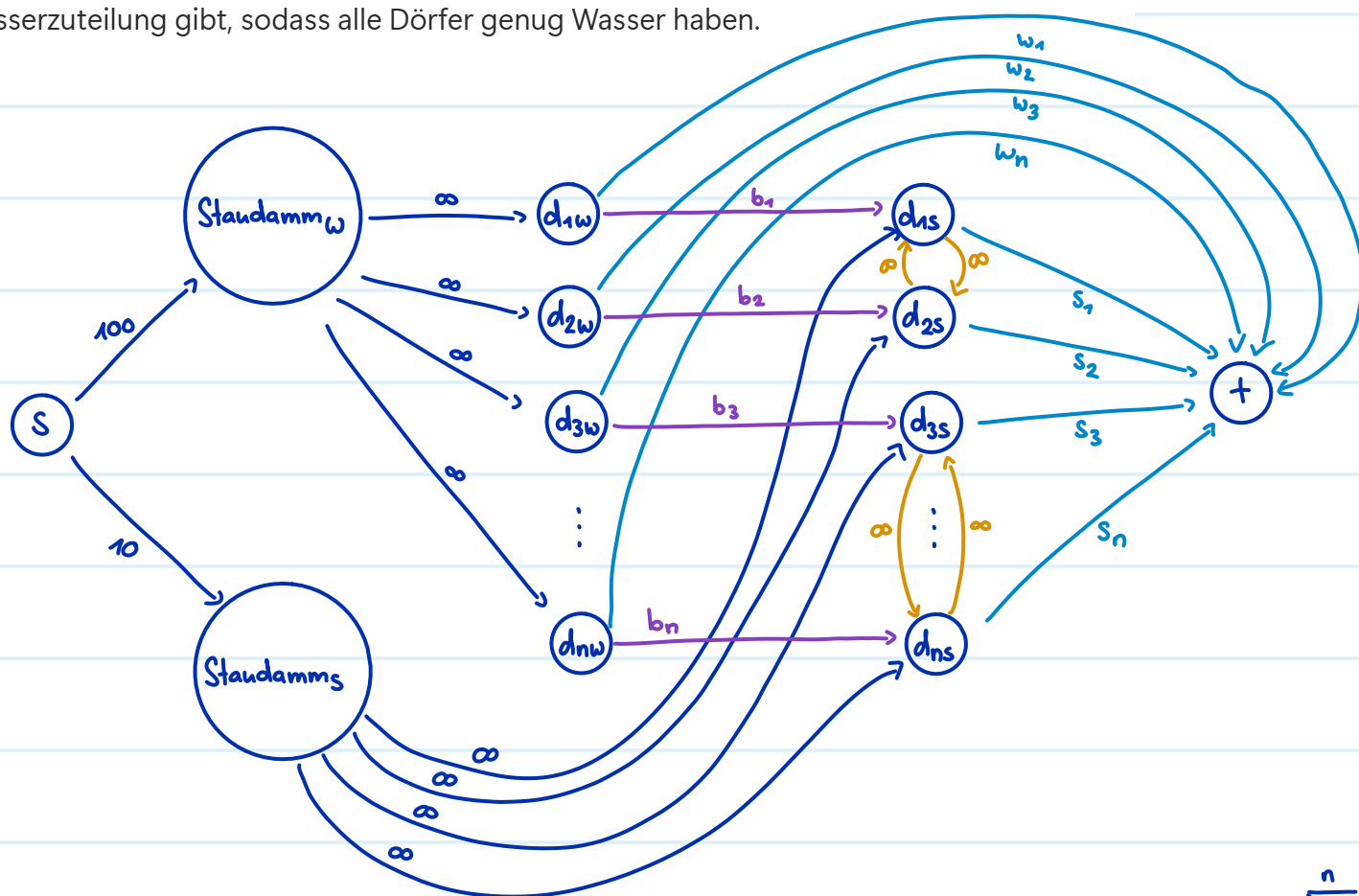


Ein Staudamm im Gebirge versorgt über ein Leitungssystem n Dörfer $d_1, \dots, d_n$ mit Wasser. Es gibt eine separate Leitung ohne Kapazitätsgrenze zu jedem Dorf. Im Winter gibt der Staudamm insgesamt 100 Einheiten Wasser, aber im Sommer nur 10 Einheiten.

Dorf i benötigt im Winter w_i Einheiten Wasser und im Sommer s_i Einheiten Wasser. Dorf i hat auch ein Speicherbecken der Grösse b_i , worin Wasser vom Winter für den Sommer aufbewahrt werden kann.

Einige Dörfer sind miteinander befreundet, d.h. sie können im Sommer gegenseitig Wasserreserven austauschen.

Modelliere dieses Problem als ein maxflow Problem, um festzustellen, ob es eine Wasserzuteilung gibt, sodass alle Dörfer genug Wasser haben.



Zuteilung möglich $\Leftrightarrow \text{maxflow} = \underbrace{\sum_{i=1}^n w_i + s_i}_{\text{Gesamtbedarf}}$

dunkelblaue Kanten = Wasser direkt vom Staudamm

hellblaue Kanten = Wasserverbrauch

violette Kanten = aufbewahrtes Wasser

orange Kanten = Hilfe zwischen befreundeten Dörfern

(Mathematische Definition hier ausgelassen)

Min-Cut Problem

Multigraph: $G = (V, E)$ ohne Schleifen, ungerichtet, ungewichtet, mehrere Kanten zwischen zwei Knoten sind erlaubt

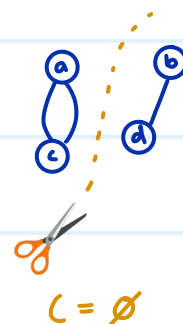
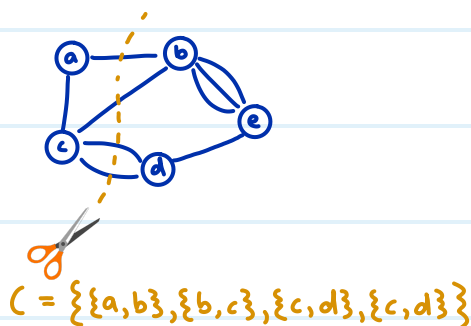
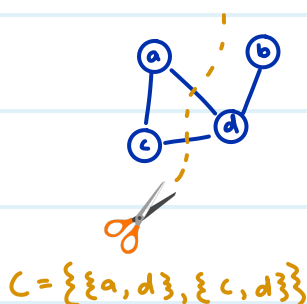
Der Grad ist die Anzahl anliegender Kanten, und nicht die Anzahl Nachbarn



$$\deg(a) = 3, \deg(b) = 5, \deg(c) = 2$$

← "C" für Cut

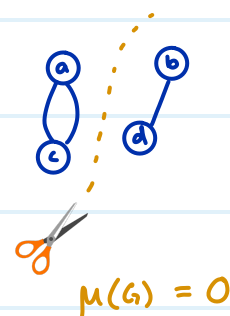
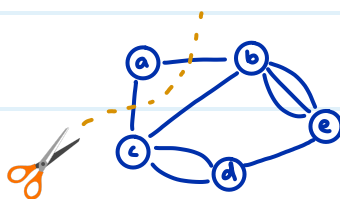
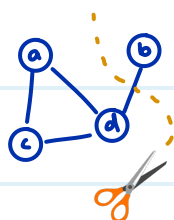
Kantenschnitt Eine Teilmenge der Kanten $C \subseteq E$, sodass $(V, E \setminus C)$ unzusammenhängend ist (= edge cut)



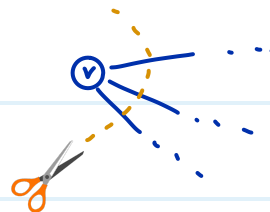
Achtung: dieser Cut ist nicht dasselbe wie der Cut bei Flüssen

← "μ" für min

$\mu(G)$: Kardinalität $|C|$ des kleinstmöglichen Schnitts C im Multigraphen G



Es gilt immer: $\mu(G) \leq \min_{v \in V} \deg(v)$
kleinster Grad



Wir könnten einfach diesen Knoten herausschneiden

Min-Cut Algorithmus via Flüsse

1) Gegeben einen Multigraphen G , konstruieren wir ein Netzwerk $N = (V, A, c, s, t)$

wobei • N dieselben Knoten hat wie G

• A hat eine Kante (u, v) und (v, u) falls G die Kante $\{u, v\}$ hat

• $c(u, v) = \text{Anzahl Kanten zwischen } u \text{ und } v \text{ in } G$

• s ist ein beliebiger, fixer Knoten

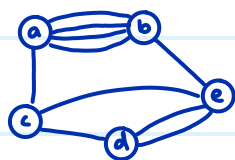
• $t \dots$ (nächster Schritt)

2) Wir iterieren über alle möglichen targets $t \in V \setminus \{s\}$ und berechnen die Kapazität

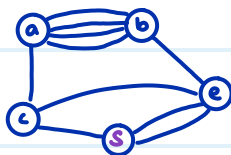
$\text{cap}(S, T)$ des minimalen Schnitts (S, T) mit dem maxflow Algorithmus

3) Gib die kleinste Kapazität aus, die gefunden wurde

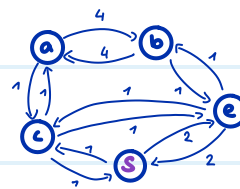
Bsp 1) G :



Wir wählen $s = d$

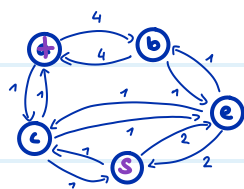


N :

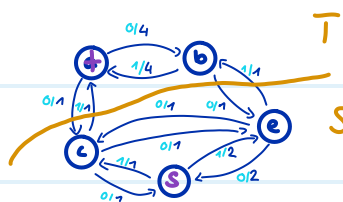


2) Wir iterieren über alle targets $t \in \{a, b, c, e\}$:

$t = a$:

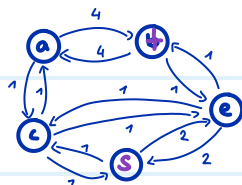


max flow Algorithmus

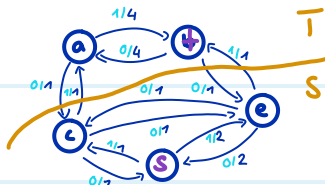


$$\text{max flow} = \min \text{cap}(S, T) = 2$$

$t = b$

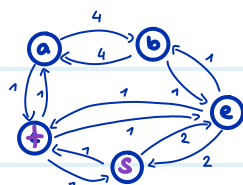


max flow Algorithmus

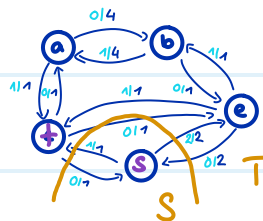


$$\text{max flow} = \min \text{cap}(S, T) = 2$$

$t = c$:

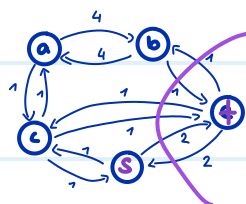


max flow Algorithmus

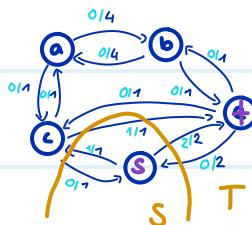


$$\text{max flow} = \min \text{cap}(S, T) = 3$$

$t = e$:



max flow Algorithmus



$$\text{max flow} = \min \text{cap}(S, T) = 3$$

$$3.) \mu(G) = \min \{2, 2, 3, 3\} = \underline{\underline{2}}$$

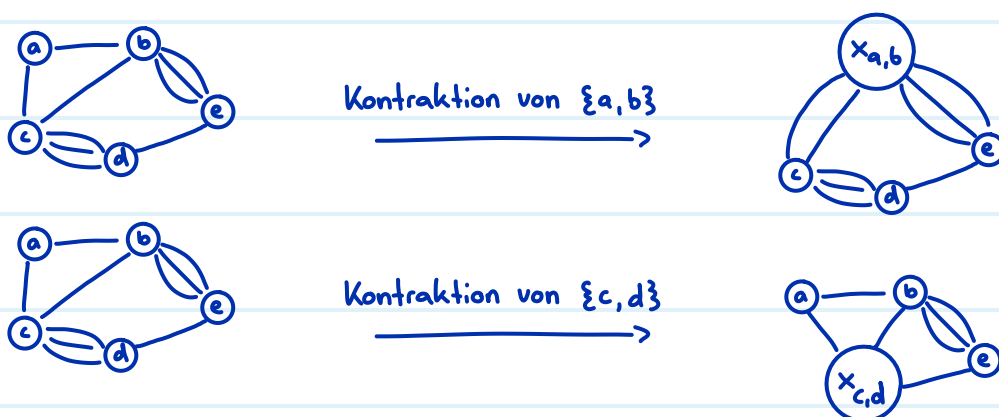
Intuition: $\text{Cap}(S, T)$ ist genau die minimale Anzahl an Kanten, die man entfernen muss, damit s und t nicht mehr verbunden sind in G

Laufzeit: $O(n \cdot \underbrace{n^3 \log n}_{\substack{\text{über } t \\ \text{iterieren}}} = O(n^4 \cdot \log n)$
 max flow in $O(m n \log n)$ mit $m \leq n^2$

Kontraktionen

Kontraktion einer Kante $e = \{u, v\}$:

- u und v werden zu einem Knoten $x_{u,v}$
- Kanten zwischen u und v werden entfernt
- andere Kanten nach u oder v gehen nun zu $x_{u,v}$
- den entstehenden Graphen nennen wir G/e

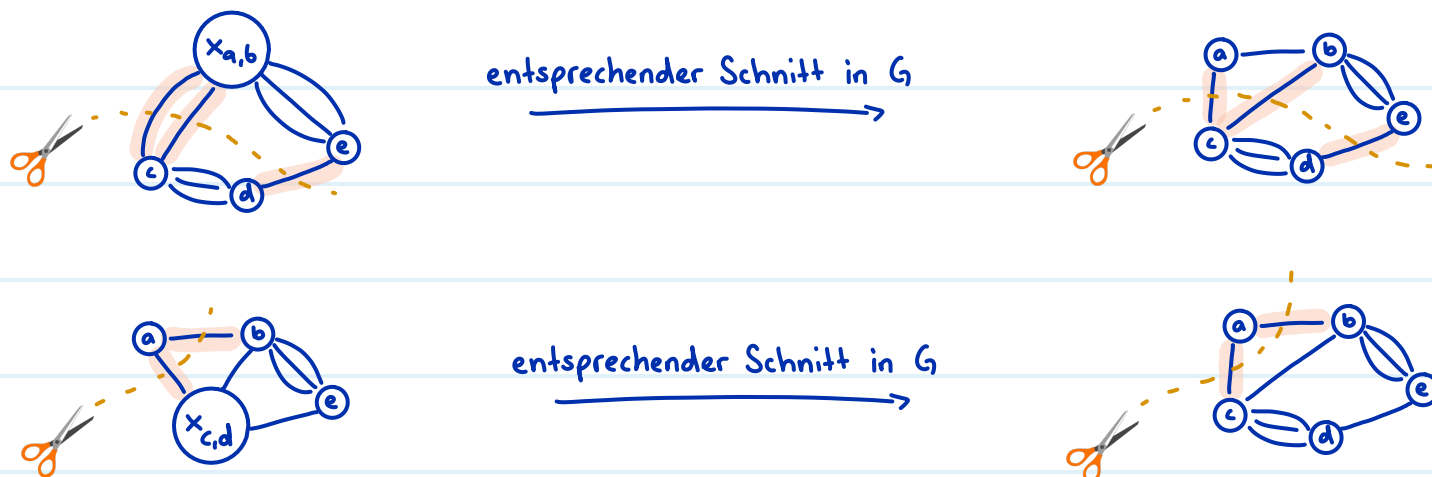


Für alle Kanten e gilt: $\mu(G/e) \geq \mu(G)$

Das heisst, durch Kontraktionen wird der minimale Kantenschnitt grösser oder er bleibt gleich.

Beweis: Sei C ein minimaler Schnitt in G/e

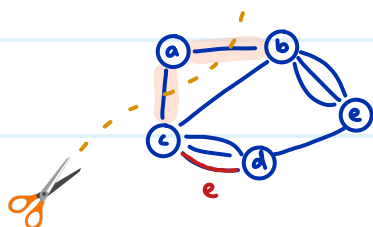
Dann gibt es auch einen Schnitt in G mit "denselben" Kanten



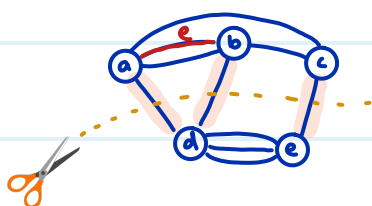
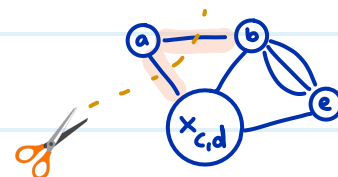
Falls G einen minimalen Kantenschnitt C mit $e \notin C$ hat, dann ist $\mu(G/e) = \mu(G)$

Beweis: Sei C ein minimaler Schnitt in G mit $e \notin C$

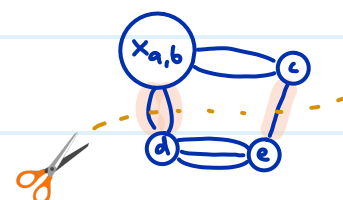
Dann gibt es auch einen Schnitt in G/e mit "denselben" Kanten



entsprechender Schnitt in G/e



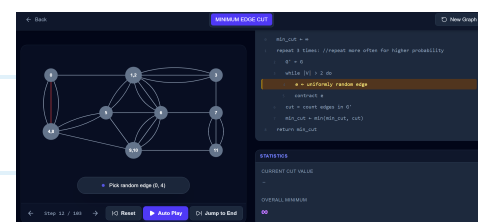
entsprechender Schnitt in G/e



Algorithmus:

$\text{CUT}(G)$ in $O(n^2)$ G zusammenhängender Multigraph

- 1: $G' \leftarrow G$
- 2: **while** $|V(G')| > 2$ **do** $\rightarrow O(n)$ Iterationen
- 3: $e \leftarrow$ gleichverteilt zufällige Kante in G'
- 4: $G' \leftarrow G'/e \rightarrow$ in $O(n)$
- 5: **return** Grösse des eindeutigen Schnitts in G'



• Monte-Carlo Algorithmus

Falls e zufällig gleichverteilt ausgewählt wird, gilt $\Pr[\mu(G) = \mu(G/e)] \geq 1 - \frac{n}{2}$

Beweis: Sei C ein minimaler Kantenschnitt und $e \in E$ zufällig gleichverteilt

$$|E| = \frac{1}{2} \sum_{v \in V} \deg(v) \quad \text{Handschlag Lemma}$$

$$\Rightarrow |E| \geq \frac{1}{2} \sum_{v \in V} |C| \quad \text{weil für alle } v \in V \text{ gilt: } \deg(v) \geq \underbrace{\mu(G)}_{=|C|}$$

$$\Rightarrow |E| \geq \frac{1}{2} n |C| \quad |V| = n$$

$$\Pr[\mu(G) = \mu(G/e)] \geq \Pr[e \notin C]$$

$$= 1 - \Pr[e \in C]$$

$$= 1 - \frac{|C|}{|E|}$$

$$\geq 1 - \frac{|C|}{\frac{1}{2} n |C|}$$

$$= 1 - \frac{2}{n}$$

schon gezeigt

Gegenwahrscheinlichkeit

zufällig gleichverteilt $\rightarrow$ Laplace Raum

$$|E| \geq \frac{1}{2} n |C|$$

kürzen

Wir definieren: $\hat{p}(G) := \Pr[\text{Cut}(G) \text{ ist korrekt}]$

(Erfolgswahrscheinlichkeit für G)

$$\text{und } \hat{p}(n) := \inf_{G=(V,E), |V|=n} \hat{p}(G)$$

(Erfolgswahrscheinlichkeit für den schwierigsten Graphen mit n Knoten)

$$\text{Es gilt } \forall n \geq 3: \hat{p}(n) \geq \frac{2}{n(n-1)}$$

Beweis Sei G ein Multigraph mit n Knoten

"Cut(G) ist korrekt" tritt genau dann ein, wenn folgende beide Ereignisse eintreten

$$E_1 := \mu(G) = \mu(G/e)$$

(= erste Kontraktion erhält den min cut)

$$E_2 := \text{Cut}(G/e) \text{ ist korrekt}$$

(= restliche Kontraktionen erhalten den min cut)

$$\text{Das heisst } \hat{p}(G) = \Pr[E_1 \wedge E_2]$$

$$= \Pr[E_1] \Pr[E_2 | E_1]$$

(Multiplikationssatz)

$$\geq \left(1 - \frac{2}{n}\right) \hat{p}(n-1)$$

($\Pr[E_1] \geq 1 - \frac{2}{n}$ wurde oben gezeigt,

und $\Pr[E_2 | E_1] \geq \hat{p}(n-1)$ per Definition)

Weil dies für jeden beliebigen Multigraphen mit n Knoten gilt, folgt:

$$\hat{p}(n) \geq \left(1 - \frac{2}{n}\right) \hat{p}(n-1)$$

Durch wiederholtes Einsetzen ergibt sich:

$$\begin{aligned} \hat{p}(n) &\geq \frac{n-2}{n} \cdot \hat{p}(n-1) && \left(1 - \frac{2}{n} = \frac{n-2}{n}\right) \\ &\geq \frac{n-2}{n} \cdot \frac{n-3}{n-1} \cdot \hat{p}(n-2) \\ &\vdots \\ &\geq \frac{n-2}{n} \cdot \frac{n-3}{n-1} \cdot \frac{n-4}{n-2} \cdot \frac{n-5}{n-3} \cdot \dots \cdot \frac{4}{6} \cdot \frac{3}{5} \cdot \frac{2}{4} \cdot \frac{1}{3} \cdot \hat{p}(2) \\ &= \frac{\cancel{n-2}}{n} \cdot \frac{\cancel{n-3}}{\cancel{n-1}} \cdot \frac{\cancel{n-4}}{\cancel{n-2}} \cdot \frac{\cancel{n-5}}{\cancel{n-3}} \cdot \dots \cdot \frac{\cancel{4}}{\cancel{6}} \cdot \frac{\cancel{3}}{\cancel{5}} \cdot \frac{\cancel{2}}{\cancel{4}} \cdot \frac{1}{3} \cdot \underbrace{\hat{p}(2)}_{=1, \text{ trivialer Fall}} \\ &= \frac{2}{n(n-1)} \end{aligned}$$

Fehlerreduktion für $\text{Cut}(G)$

• aktuelle Korrektheit: $\varepsilon \geq \frac{2}{n(n-1)}$

• Zielfehler: $\delta = e^{-\lambda}$

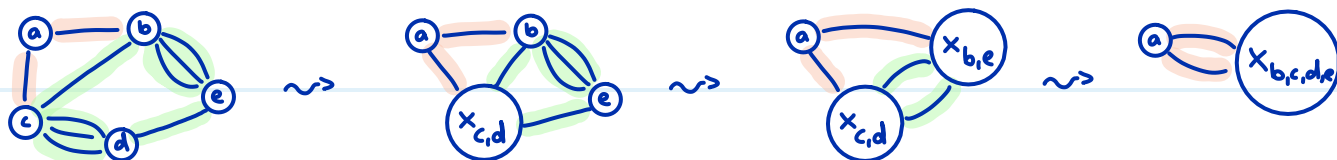
$$\Rightarrow N = \lceil \varepsilon^{-1} \ln(\delta^{-1}) \rceil = \lceil \frac{n(n-1)}{2} \ln(e^\lambda) \rceil = \lceil \lambda \frac{n(n-1)}{2} \rceil \text{ Wiederholungen, von denen wir}$$

am Schluss den kleinst gefundenen Schnitt ausgeben

Laufzeit in $O(\underbrace{n^2}_{\text{Laufzeit von Cut}(G)} \cdot \underbrace{\lambda n^2}_{\text{Wiederholungen}}) = O(\lambda n^4) \rightarrow \text{schlecht}$

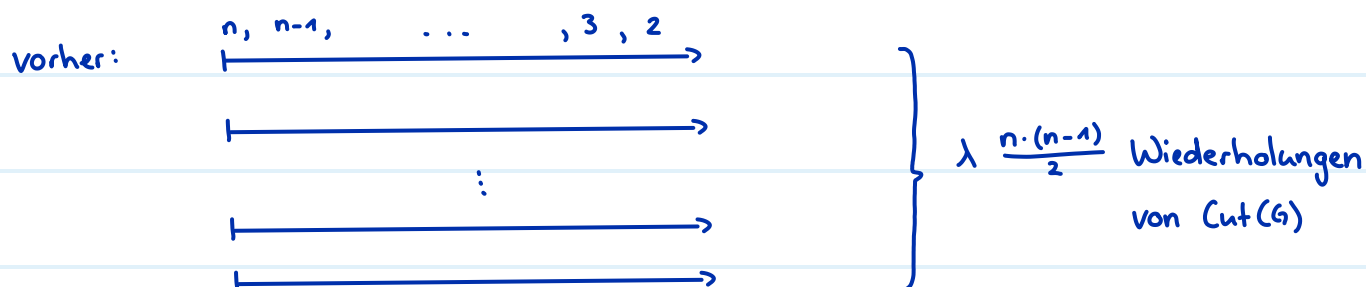
Bootstrapping

Feststellung: umso kleiner der Graph wird, desto wahrscheinlicher ist es, dass der Algorithmus einen Fehler macht

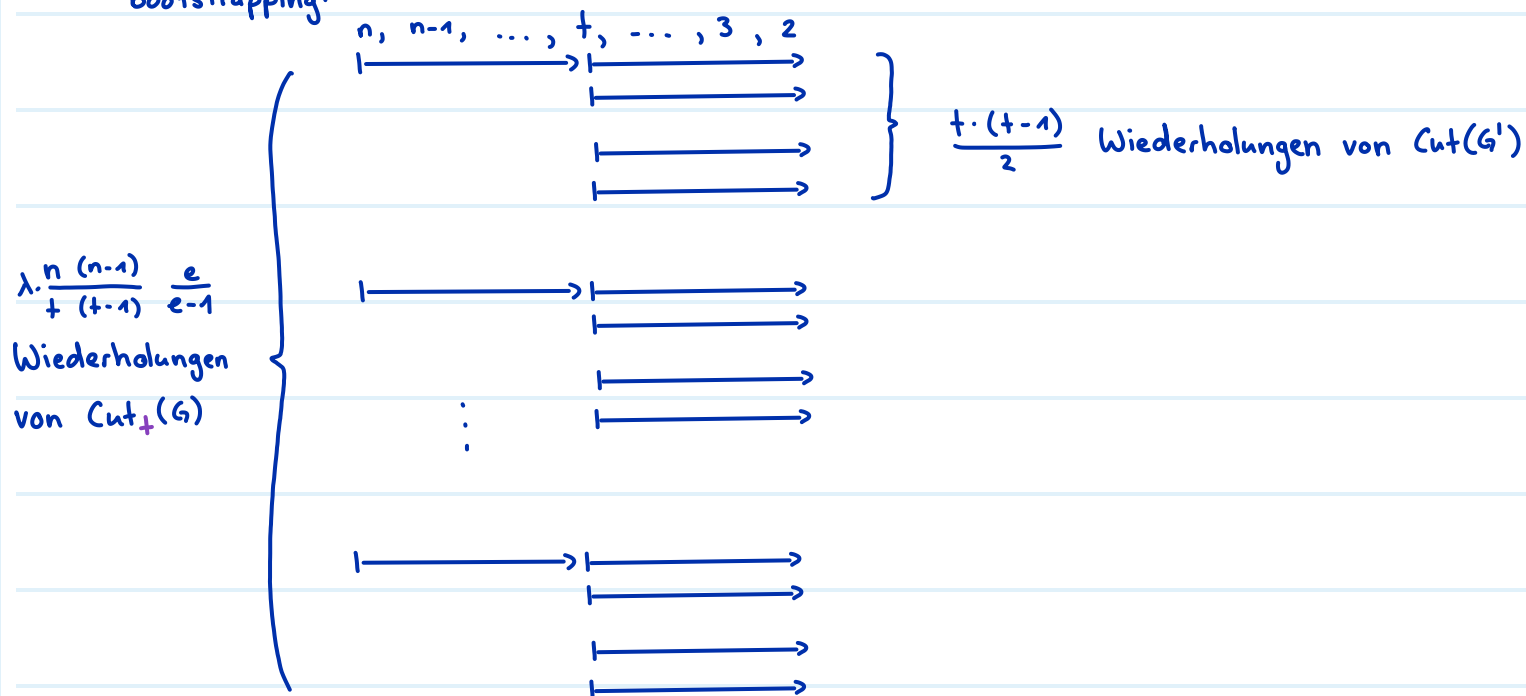


rote Kante kontrahieren = Fehler, grüne Kante kontrahieren = min cut bleibt erhalten

Idee: vorsichtiger sein wo es gefährlich ist



bootstrapping:



$\text{CUT}_t(G)$ G zusammenhängender Multigraph

- 1: $G' \leftarrow G$
- 2: **while** $|V(G')| > t$ do $\rightarrow$ bei t übrigen Knoten abbrechen
- 3: $e \leftarrow$ gleichverteilt zufällige Kante in G'
- 4: $G' \leftarrow G'/e$
- 5: **return** Resultat des $O(t^4)$ Algorithmus auf G'

$= \lambda \frac{t(t-1)}{2}$ Wiederholungen von $\text{Cut}(G')$

mit $\lambda=1 \Rightarrow$ Erfolgswahrscheinlichkeit $\geq 1 - e^{-\lambda} = 1 - e^{-1} = \frac{e-1}{e}$

Wir definieren: $\hat{p}_+(G) := \Pr[\text{Cut}_+(G) \text{ ist korrekt}]$ (Erfolgswahrscheinlichkeit für G)

und $\hat{p}_+(n) := \inf_{G=(V,E), |V|=n} \hat{p}_+(G)$

(Erfolgswahrscheinlichkeit für den schwierigsten Graphen mit n Knoten)

Analog erhalten wir

$$\hat{p}_+(n) \geq \frac{n-2}{n} \cdot \frac{n-3}{n-1} \cdot \frac{n-4}{n-2} \cdot \dots \cdot \frac{t}{t+2} \cdot \frac{t-1}{t+1} \underbrace{\hat{p}_+(t)}_{\geq \frac{e-1}{e}}$$

$$\geq \frac{n-2}{n} \cdot \frac{n-3}{n-1} \cdot \frac{n-4}{n-2} \cdot \dots \cdot \frac{t}{t+2} \cdot \frac{t-1}{t+1} \cdot \frac{e-1}{e}$$

$$= \frac{t(t-1)}{n(n-1)} \cdot \frac{e-1}{e}$$

Fehlerreduktion für $\text{Cut}_+(G)$

• aktuelle Korrektheit: $\varepsilon \geq \frac{t(t-1)}{n(n-1)} \cdot \frac{e-1}{e}$

• Zielfehler: $\delta = e^{-\lambda}$

$$\Rightarrow N = \lceil \varepsilon^{-1} \ln(\delta^{-1}) \rceil = \lceil \lambda \frac{n(n-1)}{t(t-1)} \cdot \frac{e}{e-1} \rceil \text{ Wiederholungen}$$

$$\text{Laufzeit in } O\left(\underbrace{\left(n \cdot \underbrace{(n-t)}_{\substack{\text{Anzahl Kontraktionen} \\ \text{bis } t \text{ Knoten übrig} \\ \text{bleiben}}}\right)}_{\substack{\text{Zeit für eine} \\ \text{Kontraktion}}} + t^4\right) \cdot \underbrace{\lambda \frac{n(n-1)}{t(t-1)} \cdot \frac{e}{e-1}}_{\substack{\frac{t(t-1)}{2} \text{ Wiederholungen} \\ \text{von } \text{Cut}(G') \text{ in jeweils } O(t^2)}} \leq O\left(\lambda \left(\frac{n^4}{t^2} + n^2 \cdot t^2\right)\right)$$

Wiederholungen von $\text{Cut}_+(G)$

Wie t wählen? $\rightarrow t$ ist optimal falls $\frac{n^4}{t^2} = n^2 \cdot t^2$

$$\Leftrightarrow n^4 = n^2 \cdot t^4$$

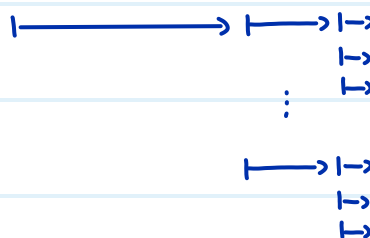
$$\Leftrightarrow n^2 = t^4$$

$$\Leftrightarrow t = \sqrt{n}$$

ergibt Laufzeit $O\left(\lambda \left(\frac{n^4}{n^2} + n^2 \sqrt{n^2}\right)\right) = O(\lambda n^3)$

Bootstrapping können wir beliebig oft machen, und erhalten im Limit die Laufzeit

$$O(n^2 \cdot \text{poly}(\log n))$$

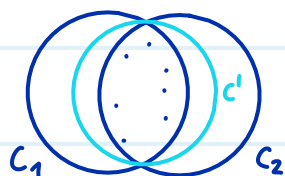


Smallest enclosing disk

Gegeben: eine endliche Punktmenge $P \subseteq \mathbb{R}^2$

Gesucht: der kleinste Kreis, der P umschließt. (Punkte auf dem Rand sind erlaubt)

Eindeutigkeit: Wir nehmen per Widerspruch an, dass $C_1 \neq C_2$ zwei kleinste umschließende Kreise sind. Dann können wir einen noch kleineren Kreis C' bestimmen, was ein Widerspruch zur Annahme ist, dass C_1 und C_2 kleinstmöglich sind.

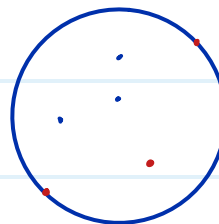
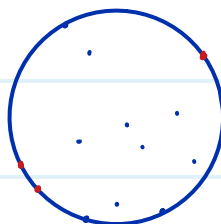
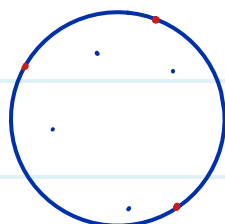


Lemma 3.26: Für jede Punktmenge P mit $|P| \geq 3$ gibt es eine Teilmenge $Q \subseteq P$,

sodass $|Q| = 3$ und $\underbrace{C(P) = C(Q)}_{\substack{\text{kleinster Kreis von } P \\ = \text{kleinster Kreis von } Q}}$

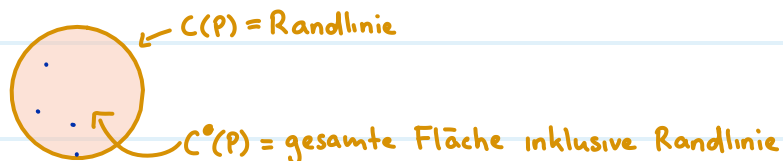
Q ist ein Zertifikat für die Minimalität des kleinsten Kreises

$Q = \text{rote Punkte}$



$\hookrightarrow Q$ muss nicht eindeutig sein

$C(P)$ vs $C^\circ(P)$



Brute Force Algorithmus:

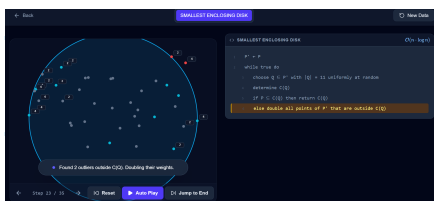
COMPLETE ENUMERATION(P) in $O(n^4)$

- 1: for all $Q \subseteq P$ mit $|Q| = 3$ do $\binom{n}{3} = \frac{n(n-1)(n-2)}{3 \cdot 2} \approx O(n^3)$ viele Möglichkeiten
- 2: bestimme $C(Q)$ in $O(1)$
- 3: if $P \subseteq C^\bullet(Q)$ then in $O(n)$, prüfen ob alle n Punkte aus P in der Kreisscheibe $C^\bullet(Q)$ enthalten sind
- 4: return $C(Q)$

Las-Vegas Algorithmus:

RANDOMISED_CLEVERVERSION(P) in $O(n \log n)$

- 1: repeat forever
- 2: wähle $Q \subseteq P$ mit $|Q| = 11$ zufällig und gleichverteilt in $O(n)$ mit Hilfe von Zählern
- 3: bestimme $C(Q)$ in $O(1)$
- 4: if $P \subseteq C^\bullet(Q)$ then in $O(n)$
- 5: return $C(Q)$
- 6: verdoppele alle Punkte von P ausserhalb von $C(Q)$



Den Beweis für die Laufzeit lassen wir aus Zeitgründen weg

Lemma 3.28: Sei P' eine Multimenge mit $|P'| = N$ Punkten

Sei $r \leq N$ und sei $R \in \binom{P'}{r}$ zufällig gleichverteilt (es gilt $|R| = r$ und $R \subseteq P'$)

Sei $X := |P' \setminus C^\bullet(R)|$ die Anzahl Punkte ausserhalb von $C(R)$

Dann ist $\mathbb{E}[X] \leq 3 \cdot \frac{N}{r+1}$

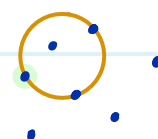
Bsp. $|P'| = 7$

$r = 4$

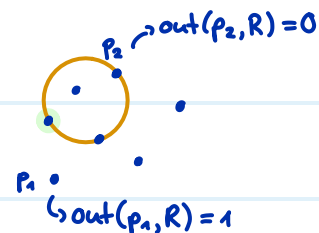
R

$C(R)$

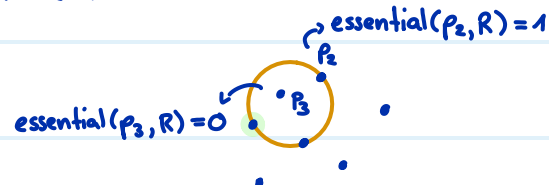
$X = 3$



Beweis: wir definieren: $\text{out}(p, R) := \begin{cases} 1 & \text{falls } p \notin C^{\circ}(R) \\ 0 & \text{sonst} \end{cases}$



$\text{essential}(p, Q) := \begin{cases} 1 & \text{falls } C(Q \setminus \{p\}) \neq C(Q) \\ 0 & \text{sonst} \end{cases}$



Für $p \notin R$ gilt: $\text{out}(p, R) = 1 \iff \text{essential}(p, R \cup \{p\}) = 1$

Sei $\Omega = \binom{P'}{r}$ ein Laplace-Raum

$$\mathbb{E}[X] = \sum_{R \in \binom{P'}{r}} \Pr[R] \mathbb{E}[X|R]$$

$$= \sum_{R \in \binom{P'}{r}} \frac{1}{\binom{N}{r}} \sum_{s \in P' \setminus R} \text{out}(s, R)$$

$$= \frac{1}{\binom{N}{r}} \sum_{R \in \binom{P'}{r}} \sum_{s \in P' \setminus R} \text{out}(s, R)$$

$$= \frac{1}{\binom{N}{r}} \sum_{R \in \binom{P'}{r}} \sum_{s \in P' \setminus R} \text{essential}(s, R \cup \{s\})$$

$$= \frac{1}{\binom{N}{r}} \sum_{Q \in \binom{P'}{r+1}} \sum_{p \in Q} \text{essential}(p, Q)$$

$$\leq \frac{1}{\binom{N}{r}} \sum_{Q \in \binom{P'}{r+1}} 3$$

$$= \frac{1}{\binom{N}{r}} \binom{N}{r+1} \cdot 3$$

$$= \frac{(N-r)! \cdot r!}{N!} \cdot \frac{N!}{(N-(r+1))! \cdot (r+1)!} \cdot 3$$

$$= 3 \frac{N-r}{r+1} \leq 3 \frac{N}{r+1}$$

Satz für bedingten Erwartungswert

$$\Pr[R] = \frac{1}{|\Omega|} = \frac{1}{\binom{N}{r}} \quad \text{und} \quad \mathbb{E}[X|R] = \sum_{s \in P' \setminus R} \text{out}(s, R)$$

zählt alle Punkte, die ausserhalb von R liegen

Distributivgesetz

weil $\text{out}(s, R) = \text{essential}(s, R \cup \{s\})$

weil $R \in \binom{P'}{r}$ plus einen weiteren Punkt $s \in P' \setminus R$ gleich $Q \in \binom{P'}{r+1}$ ist.
Wir ersetzen $R \cup \{s\}$ mit Q

es gibt ≤ 3 essential points

$$\left| \binom{P'}{r+1} \right| = \binom{N}{r+1} \quad \text{weil } |P'| = N$$

def. Binomialkoeffizient

kürzen

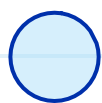
Konvexe Hülle

Definition Liniensegment: $\overline{v_0 v_1} := \{(1-\lambda) v_0 + \lambda v_1 \mid 0 \leq \lambda \leq 1\}$



Definition konvex: eine Menge C ist konvex $\stackrel{\text{def}}{\iff} \forall v_0, v_1 \in C : \overline{v_0 v_1} \subseteq C$

für beliebige zwei Punkte aus C muss das Liniensegment dazwischen auch in C sein



konvex?
→ Ja



konvex?
→ Ja



konvex?
→ Nein



konvex?
→ Nein



konvex?
→ Ja

Definition konvexe Hülle: Sei $P \subseteq \mathbb{R}^2$ eine endliche Punktmenge

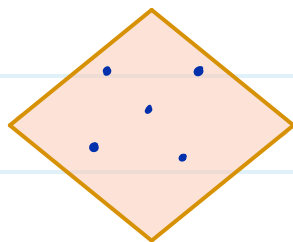
$$\text{conv}(P) := \bigcap_{\substack{P \subseteq C \subseteq \mathbb{R}^2 \\ \text{mit } C \text{ konvex}}} C$$

Schnittmenge aller konvexen Mengen, die P enthalten

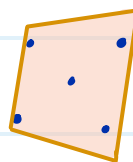
Bsp: P :



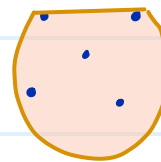
C_1 :



C_2 :

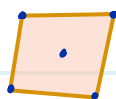


C_3 :



...

$\text{conv}(P)$:



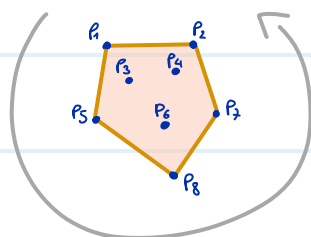
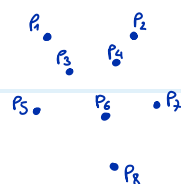
Wir können $\text{conv}(P)$ durch ein Polygon $Q = (q_0, q_1, \dots, q_{h-1})$ darstellen, dessen

h Ecken Punkte aus P sind

Reihenfolge im Gegenuhrzeigersinn

P.

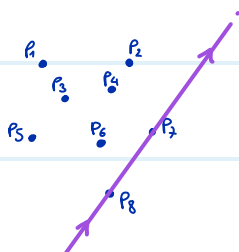
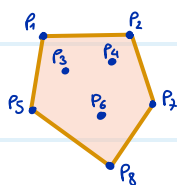
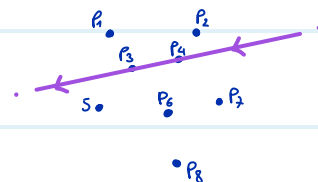
conv(P).

Polygon $Q = (p_1, p_5, p_8, p_7, p_2)$

allgemeine Lage

- keine 3 Punkte liegen auf einer Geraden
- keine 2 Punkte haben dieselbe x-Koordinate

Randkante: $(q, r) \in P^2$ mit $q \neq r$ ist eine Randkante, falls alle Punkte aus $P \setminus \{q, r\}$ links von der gerichteten Geraden durch q und r liegen

 (p_7, p_8) ist eine Randkante (p_4, p_3) ist keine Randkante

Lemma 3.34: $(q_0, q_1, \dots, q_{h-1})$ ist die Eckenfolge des conv(P) umschliessenden Polygons gegen den Uhrzeigersinn

$\Leftrightarrow$ alle Paare (q_{i-1}, q_i) für $i = 1, 2, \dots, h$ sind Randkanten (Indices mod h)

Lemma 3.35 Sei $p = (p_x, p_y)$, $q = (q_x, q_y)$, $r = (r_x, r_y)$ Punkte mit $q \neq r$

Dann liegt p links von der gerichteten Gerade durch q und r

$$\Leftrightarrow \det \begin{pmatrix} q_x - p_x & r_x - p_x \\ q_y - p_y & r_y - p_y \end{pmatrix} > 0$$

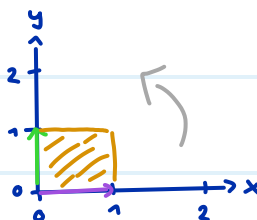
Intuition: Determinante misst die Fläche des aufgespannten Parallelograms

von a und b im Gegenuhreizersinn

$$\det\left(\begin{bmatrix} a & b \\ c & d \end{bmatrix}\right) = ad - bc$$

$$\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

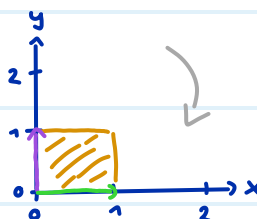
→



$$\det = 1$$

$$\begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$$

→



$$\det = -1$$

$$\begin{bmatrix} 2 & 0 \\ 1 & 1 \end{bmatrix}$$

→



$$\det = 2$$

$$\begin{bmatrix} 0 & 2 \\ 1 & 1 \end{bmatrix}$$

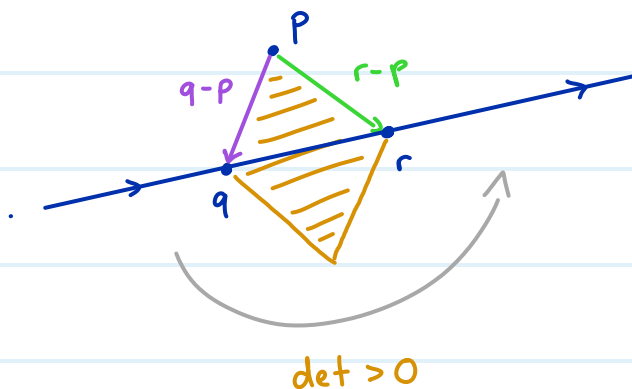
→



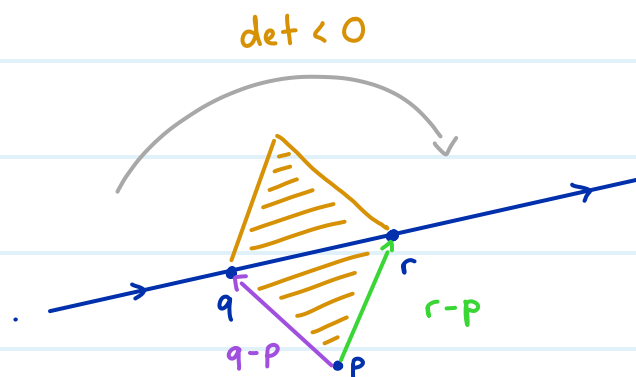
$$\det = -2$$

Vektor von Punkt A zu B = $B - A$

Fall links:

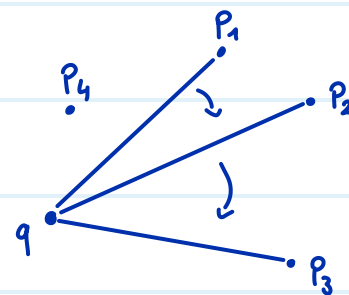


Fall rechts:



FINDNEXT(q) in $O(n)$

- 1: Wähle $p_0 \in P \setminus \{q\}$ beliebig
- 2: $q_{\text{next}} \leftarrow p_0$
- 3: **for all** $p \in P \setminus \{q, p_0\}$ **do** $\rightarrow O(n)$ Iterationen
- 4: **if** p rechts von qq_{next} **then** $q_{\text{next}} \leftarrow p \rightarrow$ in $O(1)$ mit Determinante
- 5: **return** q_{next}



Ordnungsrelation $p_1 <_q p_2$ $\stackrel{\text{def.}}{\iff} p_1$ liegt rechts von qp_2
 $\underbrace{\hspace{10em}}_{= \text{gerichtete Gerade durch } q \text{ und } p_2}$

Lemma 3.36: Sei q eine Ecke der konvexen Hülle von P

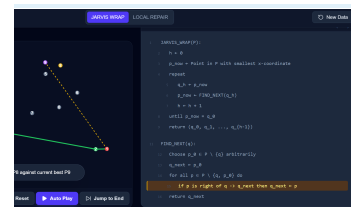
Dann ist $<_q$ eine totale Ordnungsrelation auf $P \setminus \{q\}$

total: $\forall a, b \in P \setminus \{q\}: a <_q b \vee b <_q a$

Und $(q, p_{\min})$ ist eine Randkante, wobei $p_{\min}$ das Minimum von $<_q$ ist.

JARVISWRAP(P) in $O(n \cdot h)$

- 1: $h \leftarrow 0$
- 2: $p_{\text{now}} \leftarrow$ Punkt in P mit kleinster x -Koordinate $\rightarrow$ ist immer garantiert ein Eckpunkt
- 3: **repeat** $\rightarrow h$ Iterationen
- 4: $q_h \leftarrow p_{\text{now}}$
- 5: $p_{\text{now}} \leftarrow \text{FINDNEXT}(q_h) \rightarrow$ in $O(n)$
- 6: $h \leftarrow h + 1$
- 7: **until** $p_{\text{now}} = q_0$
- 8: **return** $(q_0, q_1, \dots, q_{h-1})$



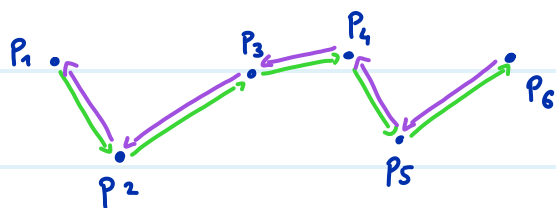
Wie gross ist h ?

- $h \leq n$

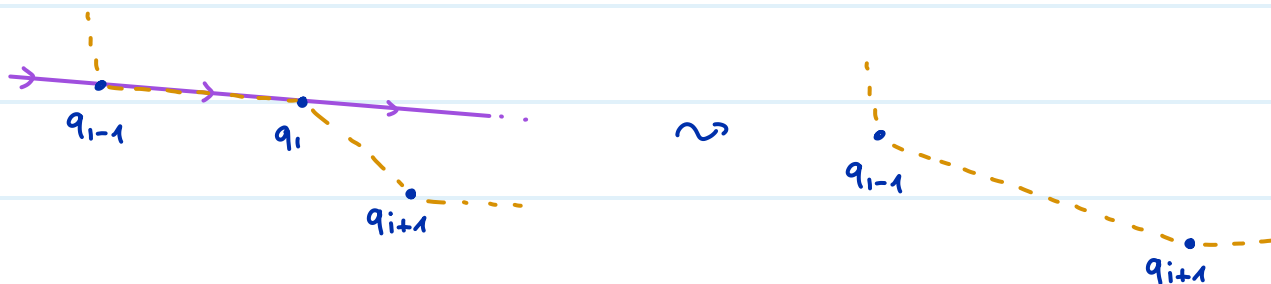
- Für zufällig gleichverteilte Punkte aus einem Quadrat: $\mathbb{E}[h] = O(\log n)$

Local Repair

- Zuerst sortieren wir P nach x -Koordinate aufsteigend: $P = (p_1, p_2, \dots, p_n)$
- Wir beginnen mit dem Polygon $(p_1, p_2, \dots, p_{n-1}, p_n, p_{n-1}, \dots, p_3, p_2)$
- p_1 und p_n sind garantiert Eckpunkte der konvexen Hülle von P
- Wir können in zwei Teilpolygonzüge aufteilen:
 - **unterer Teilpolygonzug** $(p_1, p_2, \dots, p_n)$, der x -monoton wachsend ist und keine Punkte aus P unter sich hat
 - **oberer Teilpolygonzug** $(p_n, p_{n-1}, \dots, p_1)$, der x -monoton fallend ist und keine Punkte aus P über sich hat



lokaler Verbesserungsschritt Wir betrachten drei Punkte q_{i-1} , q_i , q_{i+1} und entfernen q_i aus der Folge, falls q_{i+1} rechts von $q_{i-1}q_i$ liegt



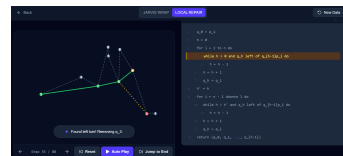
LOCALREPAIR($p_1, p_2, \dots, p_n$) in $O(n)$ BEI SORTIERTER EINGABE

▷ setzt $(p_1, p_2, \dots, p_n)$, $n \geq 3$, nach x-Koordinate sortiert, voraus

```

1:  $q_0 \leftarrow p_1$ 
2:  $h \leftarrow 0$ 
3: for  $i \leftarrow 2$  to  $n$  do           ▷ untere konvexe Hülle, links nach rechts
4:   while  $h > 0$  und  $q_h$  links von  $q_{h-1}p_i$  do
5:      $h \leftarrow h - 1$  → Verbesserung
6:    $h \leftarrow h + 1$ 
7:    $q_h \leftarrow p_i$            ▷  $(q_0, \dots, q_h)$  untere konvexe Hülle von  $\{p_1, \dots, p_i\}$ 
8:  $h' \leftarrow h$ 
9: for  $i \leftarrow n - 1$  downto  $1$  do   ▷ obere konvexe Hülle, rechts nach links
10:  while  $h > h'$  und  $q_h$  links von  $q_{h-1}p_i$  do
11:     $h \leftarrow h - 1$  → Verbesserung
12:   $h \leftarrow h + 1$ 
13:   $q_h \leftarrow p_i$ 
14: return  $(q_0, q_1, \dots, q_{h-1})$  ▷ Ecken der konvexen Hülle, gg. Uhrzeigersinn

```



Am Anfang: $2(n-1)$ Ecken

Am Ende: h Ecken

} $\Rightarrow 2(n-1) - h \leq O(n)$ lokale Verbesserungen

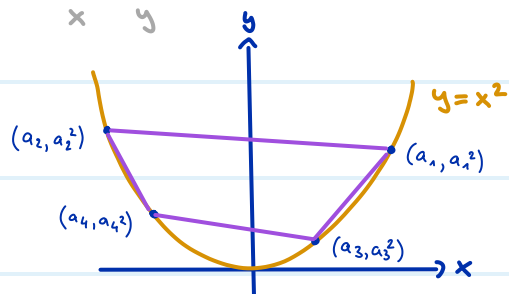
Und für jeden Punkt p_i zwei erfolglose Tests $\leq O(n)$
 einer pro Teilzug

$\Rightarrow$ Laufzeit $O(n)$

Untere Schranke $O(n \log n)$ für convex hull

Wir wollen das Array $(a_1, a_2, \dots, a_n)$ sortieren

Sei $P = \{(a_i, a_i^2) \mid 1 \leq i \leq n\}$



Ecken von $\text{conv}(P)$ im Gegenuhrzeigersinn

= Sortierung des Arrays

Aufgaben

Sei S eine (möglicherweise unendliche) Teilmenge von $\mathbb{R}^2$.

1. Wenn S endlich ist, dann ist $\text{conv}(S)$ endlich.

☐ Wahr

$\text{conv}(S)$ ist nur endlich falls $|S| \leq 1$

☒ Falsch

2. Der Rand von $\text{conv}(S)$ ist immer ein Polygon. (ein Polygon muss mindestens 3 Kanten haben)

☐ Wahr

☒ Falsch

counterexample: $|S| = 2$

3. Angenommen, dass S endlich ist und mindestens die Größe $|S| \geq 3$ hat, dann ist es möglich, dass $\text{conv}(S)$ ein Liniensegment ist.

☒ Wahr

☐ Falsch



4. Falls Algorithmus A das ConvexHull-Problem in $O(\log n)$ löst, können wir damit in $O(\log n)$ sortieren.

☐ Wahr

P erstellen dauert $O(n)$

☒ Falsch

5. Falls Algorithmus A das ConvexHull-Problem in $O(n)$ löst, können wir damit in $O(n)$ sortieren.

☒ Wahr

☐ Falsch

Seien P und Q zwei endliche Teilmengen des $\mathbb{R}^2$.

6. Für jede endliche Punktmenge P gilt, dass $x \in \text{conv}(P)$ für alle $x \in P$.

☒ Wahr per Definition

☐ Falsch

7. $P \subseteq Q \implies \text{conv}(P) \subseteq \text{conv}(Q)$

☒ Wahr

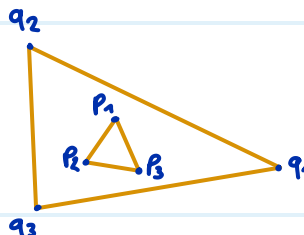
☐ Falsch

8. $\text{conv}(P) \subseteq \text{conv}(Q) \implies P \subseteq Q$

☐ Wahr

☒ Falsch

counterexample:

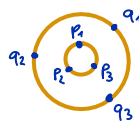


9. $C^\bullet(P) \subseteq C^\bullet(Q) \implies P \subseteq Q$

☐ Wahr

☒ Falsch

counterexample:



10. $P \subseteq Q \implies C^\bullet(P) \subseteq C^\bullet(Q)$

☒ Wahr

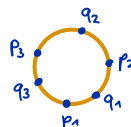
☐ Falsch

11. $C(P) \subseteq C(Q) \implies P \subseteq Q$

☐ Wahr

☒ Falsch

counterexample:

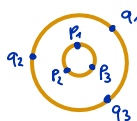


12. $P \subseteq Q \implies C(P) \subseteq C(Q)$

☐ Wahr

☒ Falsch

counterexample:



$$P = \{p_1, p_2, p_3\}$$

$$Q = P \cup \{q_1, q_2, q_3\}$$

13. $\text{conv}(P) \subseteq \text{conv}(Q) \implies C^\bullet(P) \subseteq C^\bullet(Q)$

☒ Wahr

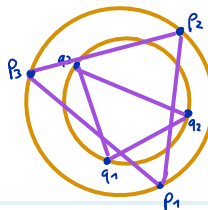
☐ Falsch

14. $C^\bullet(P) \subseteq C^\bullet(Q) \implies \text{conv}(P) \subseteq \text{conv}(Q)$

☐ Wahr

☒ Falsch

counterexample:



16. Alle Punkte aus P , die auf dem Rand $C(P)$ des kleinsten umschließenden Kreises liegen, sind auch Eckpunkte der konvexen Hülle $\text{conv}(P)$.

☒ Wahr

☐ Falsch

17. Es gilt immer $\text{conv}(P) \subseteq C^\bullet(P)$.

☒ Wahr

☐ Falsch

18. Wenn man die konvexe Hülle einer Punktmenge kennt, kann man ihren kleinsten umschließenden Kreis $C(P)$ berechnen, indem man ausschließlich die Eckpunkte von $\text{conv}(P)$ betrachtet.

☒ Wahr

☐ Falsch